

易错题合集解析

1. if-else 配对规则

题目：为了避免在嵌套的 if-else 语句中产生二义性，C语言规定与 else 子配对是()

- (A) 与其在同一行上的 if 子句
- (B) 在其之后最近的不带 else 的 if 子句
- (C) 与其缩排位置相同的 if 子句
- (D) 在其之前最近的不带 else 的同层 if 子句

[参考答案]: D

解析：

C语言中的 "else 悬空问题" (dangling else problem) 通过明确的语法规则解决：

- **配对原则**：else 总是与 同一层中、之前最近的、未配对的 if 配对
- **与格式无关**：配对关系与代码缩进、对齐方式无关，完全由语法规则决定
- **避免二义性**：编译器按照这一规则解析，不会产生歧义

示例说明：

```
1  if (a > 0)
2      if (b > 0)
3          printf("Both positive");
4  else
5      printf("What does this belong to?");
```

按照C语言规则，else 与第二个 if 配对（即内层的 if），而不是第一个 if。如果需要改变这种默认配对，必须使用花括号：

```
1  if (a > 0) {
2      if (b > 0)
3          printf("Both positive");
4  } else {
5      printf("a is not positive");
6  }
```

2. while 循环与后置递减

题目：

```
1 #include <stdio.h>
2 int main()
3 {
4     int y = 10;
5     while(y--);
6     printf("y=%d\n", y);
7     return 0;
8 }
```

程序执行后的输出结果是()。

- (A) y=0
- (B) y=-1
- (C) y=1
- (D) while 构成无限循环

[参考答案]: B

解析：

本题考察 **while** 循环条件中的**后置递减运算符** 和 **空循环体**：

1. 循环条件分析：

- `while(y--)` 使用**后置递减运算符**
- 先判断 `y` 的当前值（是否为0），然后将 `y` 减1
- 当 `y` 为 0 时，条件为假（0），循环结束，但 `y` 仍会执行减1操作

2. 循环执行过程：

```
1 初始值：y = 10
2 第1次循环：条件判断 y=10 (真) → y减为9
3 第2次循环：条件判断 y=9 (真) → y减为8
4 ...
5 第10次循环：条件判断 y=1 (真) → y减为0
6 第11次判断：条件判断 y=0 (假) → 循环结束 → y减为-1
```

注意：即使条件为假，后置递减仍然执行！

3. 特别注意：

- `while(y--);` 末尾的分号表示**空循环体**
- 循环结束后，`y` 的最终值为 **-1**

4. 常见错误：

- 误以为循环在 `y=0` 时停止，`y` 保持为 0

- 误把后置递减当作前置递减来理解

3. 嵌套循环与空循环体

题目：

```
1 #include <stdio.h>
2 int main()
3 {
4     for(int i = 0; i < 4; i++, i++)           // 外层循环
5         for(int k = 1; k < 3; k++);          // 内层循环，循环体为空语句（分
号）
6     printf("*");                             // 独立于循环之外的输出语句
7     return 0;
8 }
```

该程序段的输出结果是()。

```
1 (A) *
2 (B) *****
3 (C) **
4 (D) *****
```

[参考答案]: A

解析：

本题考察 循环控制、空循环体 和 代码结构理解：

1. 代码结构分析：

- 外层 for 循环没有使用花括号 {}
- 根据C语言语法，没有花括号时，循环体只包含紧随其后的一条语句
- 因此，内层 for 循环是外层循环的唯一循环体
- `printf("*");` 语句不属于任何循环，是独立语句

2. 循环执行分析：

- 外层循环： `for(int i = 0; i < 4; i++, i++)`
 - `i++, i++` 每次循环 `i` 增加 2
 - 循环次数： `i=0, i=2` 共2次（当 `i=4` 时条件不满足）
- 内层循环： `for(int k = 1; k < 3; k++);`
 - 末尾的分号表示空循环体
 - 实际效果： `k` 从1递增到2，循环2次，不执行任何操作

3. 关键细节：

- 内层循环的循环体是**空语句**（只有一个分号）
- `printf("*");` 在外层循环结束后执行，**仅执行一次**
- 整个程序只输出 **一个星号**

4. 常见错误:

- 误以为 `printf("*");` 是内层循环的循环体
- 错误计算外层循环次数（注意每次递增2）
- 忽略分号代表空语句这一重要语法点